

iQOO HACKATHON 2026 · GRAND FINALE APPLICATION

Festival Guardian

Predict. Respond. Relay.

Point your phone at a crowd and get a live stampede-risk score. If the network jams, alerts hop from phone to phone.

TEAM FALLING STARS

Poornachandra P M · Dushyanth M · M Sujay



Crowds turn deadly in minutes



Nobody sees it building

Density rises gradually. By the time it feels dangerous, exits are already blocked.



Networks fail at peak

Thousands of phones in one spot overload towers, so calls and messages fail.



Help arrives late

Organizers get no early signal and no location, so response starts after the crush.

Festival grounds, temple queues, railway platforms.

Point. Read. Warn.



Point the camera

On-device detection, in-browser today — no upload, no cloud call.



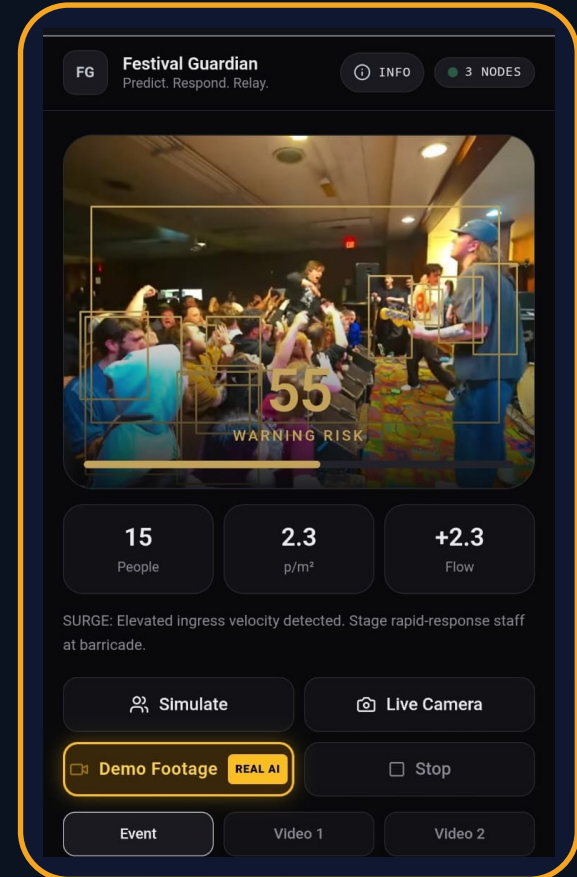
Live risk meter

Density and flow combine into one clear score.



Early warning

Organizers are alerted before a crush forms.



Real detection on real crowd footage — not synthetic data

How it works, today vs. the Finale build

1



Camera

Live feed from the phone

2



Detect

TensorFlow.js (COCO-SSD) runs on-device, in-browser, over WebGL

3



Score

Density + flow become a 0-100 risk

4



Alert

A typed Alert object is created, with TTL + dedup

5



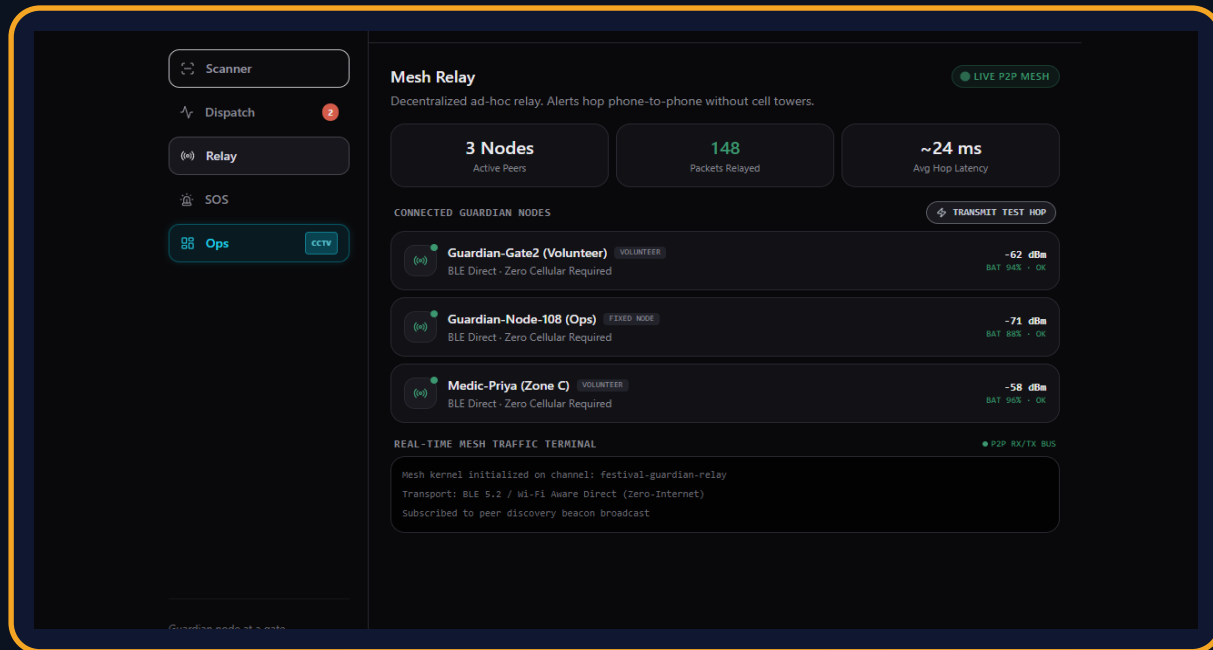
Relay

Working today same-device; native phone-to-phone at the Finale

Today: real on-device detection, in a web prototype. Finale: ported to native Android + Nearby Connections for true phone-to-phone offline relay.

Relay engine — real code, local sim today

TTL hop bounds, packet dedup and a live node list are real and working. The transport shown here is a same-device simulation; true phone-to-phone hops ship at the Finale.



Today: relay engine + telemetry UI verified same-device. Finale: Android Nearby Connections, two real phones, one offline.

Who gets the alert



Event organizers

Live risk score, location and photo on their phone.



Emergency services (108 / 112)

A pre-filled emergency call with the exact location.



Family and nearby volunteers

Trusted contacts and nearby app users get the alert and pin.

Roadmap: from node to network



Phase 1 — today

One app, one role. A Guardian node (camera, risk meter, SOS) built and demoed.



Phase 2 — split by role

Separate Guardian (node) and Dispatch (organizer) views: a map of zones, live risk per gate, the alert feed with hop/source — no camera needed for Dispatch.



Phase 3 — bring your own cameras

The same detection pipeline reads an RTSP/CCTV feed instead of a phone camera — venues use hardware they already have, no rebuild required.

Phase 1 is what we built and demo today. Phases 2-3 are the near-term extension, not a rebuild — same detection pipeline, new input/output.

Today vs. the Finale build



Built today (web prototype)

- Real on-device detection: TF.js + COCO-SSD over WebGL
- Alert engine: typed alerts, TTL, dedup — all working
- Relay verified same-device (browser tabs)



Finale: Red Light (phone only)

- Port detection to TFLite, run on the Snapdragon NPU
- Android Nearby Connections for real phone-to-phone hops
- Live demo: 2-3 iQOO phones, one offline

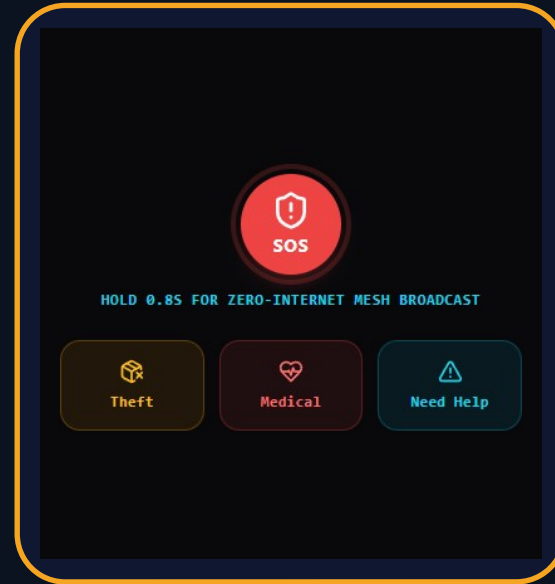
Scored on: phone-first execution, AI integration, Office Kit, real-world relevance, pitch.

Two views, one app: Guardian + Dispatch

A volunteer at a gate sees only their node. An organizer sees the aggregate — venue density, active incidents, and a chokepoint focus across zones. Names, counts and incidents are demo data.



Organizer Control Room — Dispatch view



Guardian node — SOS / Theft / Medical

48 hours: what we ship

0-12h

Detection + meter

On-device counting and a live risk score

12-30h

Offline relay

Two-phone SOS hop working end to end

30-42h

Alerts + first aid

Location, photo and a voice-guided first-aid card

42-48h

Polish + pitch

Demo rehearsal, backup video, final deck

Festival Guardian: predict, respond, relay.

Thank You